

1 awake robot Arm

서보를 하나씩 움직여 어느 관절인지 확인한다

코드 - 01_servo_single_test.ino

```
#include <Servo.h>
int PIN = 2;           // ★ 이 숫자만 2→3→9→8→4→5 로 바꿔 업로드
Servo myservo;

void setup() {
  myservo.attach(PIN); // PIN 번호에 서보 연결
}
void loop() {
  for (int a=0; a<=90; a++){ myservo.write(a); delay(15); } // 0→90도
  for (int a=90; a>=0; a--){ myservo.write(a); delay(15); } // 90→0도
}
```

코드 설명

- **attach(PIN)** : 서보를 그 핀에 연결한다. PIN 숫자만 바꿔 업로드하면 다른 서보가 움직인다.
- **write(a)** : 서보를 a도로 보낸다. for문으로 0 → 90 → 0을 반복해 왕복시킨다.
- **delay(15)** : 한 칸 움직일 때마다 잠깐 멈춤 → 숫자를 키우면 더 천천히.
- 움직이는 서보를 보며 **관절 매핑표**(핀=관절)를 채운다.

2 Code Dissection

기본 조이스틱 제어 코드를 해석한다

코드 - 04_basic_joystick_control.ino

```
const int SERVOS = 6;                // 서보 개수
int PIN[SERVOS], MIN[SERVOS], MAX[SERVOS], INITANGLE[SERVOS], ANA[SERVOS];

void setup(){
  PIN[0]=2; MIN[0]=0; MAX[0]=180; INITANGLE[0]=90; ANA[0]=0; // 1번
  ... (6번까지)
  for(int i=0;i<SERVOS;i++){ myservo[i].attach(PIN[i]);
    myservo[i].write(INITANGLE[i]); }
}

void loop(){
  value = analogRead(ANA[i]);          // 조이스틱 값(0~1023)
  if(value>612) currentAngle++;         // 한쪽 → 각도 +
  else if(value<412) currentAngle--;    // 반대 → 각도 -
}
```

코드 설명

- 코드는 **3덩어리** — 준비(변수) · setup(설정) · loop(반복).
- **해석표에서 찾을 값** : 서보 개수(SERVOS), 핀(PIN), 조이스틱(ANA), 시작각(INITANGLE), 최소·최대(MIN/MAX).
- **612 / 412** : 조이스틱 값의 경계. 그 사이(가운데)는 '멈춤'.
- 숫자를 바꿔 보며(코드 탐정) 어떤 값이 무엇을 바꾸는지 확인한다.

3 Joystick Master

정밀 제어의 원리를 이해한다

 코드 - 04_basic_joystick_control.ino

```
void loop(){
  delay(50); // 전체 속도(작을수록 빠름)
  for(int i=0;i<SERVOS;i++){
    value[i] = analogRead(ANA[i]);
    if(value[i]>612){
      if(currentAngle[i]<MAX[i]) currentAngle[i]++; // 한 번에 +1도씩만
      myservo[i].write(currentAngle[i]);
    } else if(value[i]<412){
      if(currentAngle[i]>MIN[i]) currentAngle[i]--; // -1도씩만
      myservo[i].write(currentAngle[i]);
    }
  }
}
```

 코드 설명

- 각도를 **한 번에 1도씩만** 바꾼다 → 부드럽고 안정적인 움직임.
- 한 번에 크게 움직이면 팔이 흔들리거나 물체를 놓친다.
- **delay(50)**을 줄이면 빨라지지만 거칠어진다 → 정밀함과 속도는 트레이드오프.
- MIN/MAX가 각도를 벗어나지 못하게 막아 안전을 지킨다.

4 Home Position

전원을 켤 때의 안정 자세를 정한다

 코드 - 04_basic_joystick_control.ino

```
void setup(){
  //          핀   최소   최대   시작각   조이스틱
  PIN[0]=2; MIN[0]=0; MAX[0]=180; INITANGLE[0]=90; ANA[0]=0;
  ...
  PIN[5]=5; MIN[5]=0; MAX[5]=90;  INITANGLE[5]=45; ANA[5]=5; // 그리퍼

  for(int i=0;i<SERVOS;i++){
    myservo[i].attach(PIN[i]);
    myservo[i].write(INITANGLE[i]);  // ← 켜면 이 각도로 이동
  }
}
```

 코드 설명

- **INITANGLE** = 전원을 켰을 때 가는 시작 자세(홈 포지션).
- 이 값을 바꿔 팔이 안정적으로 서는 자세를 직접 정한다.
- 그리퍼(6번)는 가동범위가 0~90이라 시작각도 그 안(예: 45)이어야 한다.
- 홈이 안정적이면 매번 같은 자세에서 작업을 시작할 수 있다.

5 gripper craftsman

그리퍼(손)를 여닫아 물체를 집는다

 코드 - 08_functions_button.ino

```
const int GRIPPER = 5;           // 그리퍼 = 6번 서보(인덱스 5)
const int GRIP_OPEN  = 0;        // 열림 0도
const int GRIP_CLOSE = 90;       // 닫힘 90도

void gripperOpen(){ myservo[GRIPPER].write(GRIP_OPEN); delay(400); }
void gripperClose(){ myservo[GRIPPER].write(GRIP_CLOSE); delay(400); }
```

 코드 설명

- 그리퍼는 **0도(열림) / 90도(닫힘)** 두 각도만 쓰면 된다.
- **delay(400)** : 손이 다 움직일 때까지 기다린다(안 그러면 다음 동작이 먼저 시작).
- 무른 물체(마시멜로)는 각도를 조금 덜 닫아 힘을 줄인다.
- 이렇게 손 제어를 **함수**로 묶어 두면 다음 단계에서 재사용한다.

6 Talking to a Robot

시리얼로 현재 각도를 읽어 기록한다

 코드 - 07_serial_record.ino

```
void setup(){
  Serial.begin(9600);          // ★ 컴퓨터와 통신 시작
  ...
}
void loop(){
  // (조이스틱 제어) ...
  if(Serial.available()>0 && Serial.read()=='p') printAngles();
}
void printAngles(){            // 6축 각도를 한 줄로 출력
  for(int i=0;i<SERVOS;i++) Serial.print(my servo[i].read());
}
```

 코드 설명

- **Serial.begin(9600)** : 로봇과 컴퓨터가 '대화'할 통로를 연다.
- 시리얼 모니터에 **p**를 입력하면 현재 6축 각도가 숫자로 출력된다.
- 즉 로봇의 자세를 **숫자(데이터)**로 남기는 것 — 자동화의 재료가 된다.
- 출력된 각도를 활동지 포즈 카드에 그대로 적는다.

7 Pick & Place

집기 → 이동 → 놓기를 순서로 나눈다

 코드 - 08_functions_button.ino

```
void moveTo(int target[]){    // 팔(1~5번)을 목표 자세로 부드럽게
    ... 1도씩 target 으로 이동 ...
}
// 수동 작업 순서(설계):
gripperOpen();    // 손 펴고
moveTo(PICK);    // 집는 위치로
gripperClose();    // 집고
moveTo(PLACE);    // 놓을 위치로
gripperOpen();    // 놓기
```

 코드 설명

- 작업을 **집기** → **들기** → **이동** → **놓기**의 순서로 쪼갬다.
- **moveTo(target)** : 팔을 목표 자세로 옮기는 함수(그리퍼는 건드리지 않음).
- 순서만 정하면 사람이든 로봇이든 같은 작업을 반복할 수 있다.
- 이 '순서 설계'가 다음 단계 자동화의 밑그림이 된다.

8 Posture Recipe Book

핵심 자세를 각도 데이터로 저장한다

 코드 - 08_functions_button.ino

```
// 8차시에서 기록한 각도값을 그대로 넣는다
int HOME[SERVOS]      = { 90, 90, 90, 90, 90, 0 };
int PICK_READY[SERVOS] = { 60, 100, 80, 90, 90, 0 };
int PICK[SERVOS]      = { 60, 130, 110, 90, 90, 0 };
int PLACE_READY[SERVOS] = { 120, 100, 80, 90, 90, 0 };
int PLACE[SERVOS]      = { 120, 130, 110, 90, 90, 0 };
```

 코드 설명

- 성공한 자세를 **각도 배열**로 저장한다(HOME/PICK/PLACE ...).
- 같은 값을 넣으면 **항상 같은 자세**가 재현된다 → 재현성.
- 이 포즈 데이터가 자동 동작의 '레시피(재료)'가 된다.
- 6번(그리퍼) 자리는 moveTo가 무시하고 따로 제어한다.

9 Create a function

반복 동작에 이름을 붙여 함수로 만든다

 코드 - 08_functions_button.ino

```
// 만드는 법: void 이름(){ 할 일 } / 쓰는 법: 이름();  
void gripperOpen(){ myservo[5].write(0); delay(400); }  
void gripperClose(){ myservo[5].write(90); delay(400); }  
void goHome(){  
  for(int i=0;i<SERVOS;i++) myservo[i].write(INITANGLE[i]);  
}  
// 부를 때: goHome(); gripperClose();
```

 코드 설명

- **함수** = 자주 쓰는 동작 덩어리에 '이름'을 붙인 것.
- 만드는 3요소 : ① 이름 ② void(돌려줄 값 없음) ③ { 할 일 }.
- **loop() 바깥**에 만들고, 어디서든 **이름()**; 한 줄로 부른다.
- 복붙 대신 함수를 쓰면 코드가 짧아지고 고치기 쉽다.

10 Automatic Pick & Place

함수를 순서대로 불러 자동 실행한다

 코드 - 08_functions_button.ino

```
void autoPickAndPlace(){          // 함수들을 '순서대로' 호출 = 자동화
  gripperOpen(); moveTo(PICK_READY); moveTo(PICK);
  gripperClose(); moveTo(PICK_READY); moveTo(PLACE_READY);
  moveTo(PLACE); gripperOpen();    goHome();
}
void loop(){                      // 버튼(디지털 7번)을 누르면 실행
  if(lastButton==HIGH && digitalRead(7)==LOW) autoPickAndPlace();
}
```

 코드 설명

- 함수를 **순서대로 호출**하면 사람이 하던 작업이 자동으로 실행된다.
- **버튼(조이스틱 SW, 7번)**을 누르는 순간 한 번 실행된다.
- 실패하면 포즈 각도값을 조금씩 조정해 성공률을 올린다.
- 이것이 '반복노동을 코드로 대체'하는 자동화의 핵심이다.

11 Final Mission Design

배운 것을 조합해 팀 미션을 설계한다

코드 - (팀이 직접 작성)

```
// 우리 팀 미션 = 지금까지 배운 조각들의 조합
// 1) 포즈 데이터 정하기 : int A[SERVO] = {...}; int B[SERVO] = {...};
// 2) 동작 함수 만들기 : void step1(){ moveTo(A); gripperClose(); }
// 3) 순서대로 부르기 : void mission(){ step1(); step2(); ... }
// 4) 버튼/시리얼로 실행하고 반복 테스트로 튜닝
```

코드 설명

- 새 코드가 아니라 **배운 조각(포즈·함수·순서·버튼)**을 조합한다.
- 미션에 맞는 포즈 배열과 동작 함수를 새로 정의한다.
- mission() 안에서 함수들을 순서대로 불러 완성한다.
- 테스트하며 각도·delay를 조정해 성공률을 높인다.

12 Demo & Career

완성 코드를 시연하고 진로와 연결한다

코드 - (종합)

```
// 최종 시연용 - 우리 팀 mission() 을 버튼으로 실행
void loop(){
  if(눌림) mission();    // 데모데이 시연
}
// 이 코드가 하는 일 = 어떤 직업의 축소판일까?
//   · 자동화 엔지니어 · 스마트팩토리 · 물류 · 재활로봇 ...
```

코드 설명

- 완성한 mission()을 버튼으로 실행해 **시연(데모데이)** 한다.
- 코드 한 줄 한 줄이 실제 산업 현장의 **어떤 일**과 닮았는지 이야기한다.
- '내가 없애고 싶은 반복노동은?'을 코드 아이디어로 발표한다.
- 활동 전체가 진로 탐색의 근거(포트폴리오)가 된다.

★ group dance mission

동작 함수를 나눠 만들어 군무를 완성한다

💻 코드 - 09_dance.ino

```
const int BEAT = 700;           // 한 박자(ms) - 모든 팀이 같은 값!  
void moveTimed(int t[], int dur){ // 거리와 상관없이 dur(ms)만에 도착  
    ... 시간 비율 f 로 보간 이동 ... // → 어떤 동작도 정확히 한 박자  
}  
void waveHand(){ moveTimed(WAVE_L,BEAT); moveTimed(WAVE_R,BEAT); }  
void routine(){ waveHand(); twist(); bow(); } // 팀이 합치는 안무
```

📖 코드 설명

- 학생마다 동작 함수(waveHand·twist·bow ...)를 하나씩 맡아 만든다.
- **moveTimed**는 이동 거리와 무관하게 정확히 한 박자에 끝난다 → 박자 동기화.
- 모든 팀이 같은 BEAT를 쓰고 동시에 시작하면 **칼군무**가 된다.
- 함수를 순서대로 부르는 routine()이 곧 안무 타임라인이다.